

[WHS4] DevSecOps Infrastructure 16반 김민서 (3625)

01. Repository

- GitHub: https://github.com/Opal1031/WHS_4th_DevSecOps_Infrastructure
- 점검 대상: `lab/homework/vulnerable/main.tf`
- 조치 결과: `lab/homework/fixed/main.tf`

02. 과제 개요

사내 웹 서비스용 AWS 인프라 코드에 남아 있는 위험한 설정을 `tfsec`로 점검하고, 기존 코드의 전체적인 구조를 유지하면서 보안 설정을 추가·수정하였다. 과제의 핵심 목표는 수정 후 재스캔에서 **CRITICAL**과 **HIGH** 취약점을 모두 0으로 만드는 것이다.

03. 실습 환경 및 스캔 명령

- 운영체제: macOS
- 컨테이너 실행 환경: Docker Desktop
- IaC: Terraform
- 보안 스캐너: tfsec

`lab` 디렉터리에서 다음 명령으로 수정 전·후 코드를 각각 스캔하였다.

```
./scan.sh tfsec homework/vulnerable  
./scan.sh tfsec homework/fixed
```

04. 수정 전 스캔 결과

```
timings
-----
disk i/o      836.208µs
parsing      246.083µs
adaptation    134.291µs
checks       3.119917ms
total        4.336499ms

counts
-----
modules downloaded  0
modules processed   1
blocks processed    6
files read          1

results
-----
passed           2
ignored          0
critical         4
high            10
medium           5
low              5

2 passed, 24 potential problem(s) detected.

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
-----
✅ 스캔 완료. 위에서 빨간 항목 (FAILED · CRITICAL · HIGH)을 확인합니다.
○ (base) kimminseo@gimminseoui-MacBookAir lab %
```

구분	개수
Passed	2
CRITICAL	4
HIGH	10
MEDIUM	5
LOW	5
잠재 문제 합계	24

수정 전 코드에서는 S3 공개 설정, 보안 그룹의 전체 인터넷 허용, RDS 공개 접근과 평문 저장, 하드코딩된 비밀번호, EBS 암호화 미적용 등이 CRITICAL 또는 HIGH 등급으로 탐지되었다.

05. 취약 설정 수정 내용과 이유

항목	수정 내용	수정 이유
신뢰 네트워크	<code>trusted_cidr</code> 변수를 만들고 기본값을 <code>10.0.0.0/8</code> 로 지정	허용 대역을 한곳에서 관리하고 인터넷 전체가 아닌 사내 망으로 접근 범위를 제한하기 위해 수정하였다.
DB 비밀번호	하드코딩된 비밀번호를 <code>sensitive</code> 변수 <code>db_password</code> 로 분리	소스 코드와 Git 기록에 인증 정보가 평문으로 노출되는 것을 방지하기 위해 수정하였다.
KMS 키	고객 관리형 KMS 키를 추가하고 자동 키 교체를 활성화	S3·RDS·EBS 암호화 키를 직접 통제하고 장기간 동일 키 사용 위험을 줄이기 위해 수정하였다.

항목	수정 내용	수정 이유
S3 ACL	<code>public-read</code> 를 <code>private</code> 으로 변경	로그 버킷의 객체와 메타데이터가 인터넷에 공개되는 것을 차단하기 위해 수정하였다.
S3 퍼블릭 차단	Public Access Block의 네 가지 옵션을 모두 <code>true</code> 로 설정	ACL이나 버킷 정책의 실수로 S3가 다시 공개되는 것을 방어하기 위해 수정하였다.
S3 암호화	고객 관리형 KMS 키를 사용하는 <code>aws:kms</code> 서버 측 암호화 추가	저장된 로그가 유출되더라도 평문으로 바로 읽히는 것을 방지하기 위해 수정하였다.
SSH 인바운드	포트 22의 <code>0.0.0.0/0</code> 을 <code>trusted_cidr</code> 로 변경	인터넷 전체에서 발생할 수 있는 SSH 스캔과 무차별 대입 공격을 줄이기 위해 수정하였다.
MySQL 인바운드	포트 3306의 <code>0.0.0.0/0</code> 을 <code>trusted_cidr</code> 로 변경	데이터베이스 포트가 외부에 직접 노출되지 않도록 접근 대역을 제한하기 위해 수정하였다.
보안 그룹 아웃바운드	전체 인터넷 송신을 <code>trusted_cidr</code> 로 제한	침해 발생 시 외부 서버로 데이터가 유출되거나 악성 통신이 이루어질 범위를 줄이기 위해 수정하였다.
RDS	공개 접근을 끄고 스토리지 암호화와 KMS 키, 보안 그룹을 적용	데이터베이스의 인터넷 노출을 막고 저장 데이터와 접근 경로를 함께 보호하기 위해 수정하였다.
EBS	볼륨 암호화와 고객 관리형 KMS 키를 적용	스냅샷 또는 볼륨이 유출되어도 저장 데이터가 평문으로 노출되지 않도록 수정하였다.

06. 수정한 `main.tf` 전체 코드

기존 리소스 배치와 주석 형식을 유지하면서 필요한 변수와 보안 리소스 및 속성만 추가하였다.

```
# =====
# 과제 · 취약한 인프라 (수업과 다른 새 시나리오)
# 상황: 사내 웹서비스 인프라를 급하게 만들었더니 곳곳이 위험합니다.
# 목표: 스캔해서 CRITICAL·HIGH 를 전부 0으로 만드세요.
# =====

provider "aws" {
  region = "ap-northeast-2"
}

# SSH·MySQL 접근을 허용할 사내망 대역
variable "trusted_cidr" {
  type    = string
  default = "10.0.0.0/8"
}

# DB 비밀번호를 코드에 직접 작성하지 않고 외부에서 입력
variable "db_password" {
  type        = string
  sensitive   = true
}

# S3·RDS·EBS 암호화에 사용할 고객 관리형 KMS 키
```

```

resource "aws_kms_key" "infrastructure" {
  description      = "KMS key for company web infrastructure"
  enable_key_rotation = true
}

# -----
# 1) 로그 저장용 S3 버킷
# -----
resource "aws_s3_bucket" "logs" {
  bucket = "company-web-logs-2026"
}

resource "aws_s3_bucket_acl" "logs_acl" {
  bucket = aws_s3_bucket.logs.id
  acl    = "private" # 수정: 로그 버킷의 공개 ACL 제거
}

# 수정: ACL·정책을 통한 모든 퍼블릭 액세스 차단
resource "aws_s3_bucket_public_access_block" "logs" {
  bucket = aws_s3_bucket.logs.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# 수정: 고객 관리형 KMS 키로 로그 데이터 암호화
resource "aws_s3_bucket_server_side_encryption_configuration" "logs" {
  bucket = aws_s3_bucket.logs.id

  rule {
    apply_server_side_encryption_by_default {
      kms_master_key_id = aws_kms_key.infrastructure.arn
      sse_algorithm      = "aws:kms"
    }
  }
}

# -----
# 2) 웹 + DB 보안 그룹
# -----
resource "aws_security_group" "web" {
  name = "web-sg"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.trusted_cidr] # 수정: SSH 접근을 사내망으로 제한
  }

  ingress {

```

```

    description = "MySQL"
    from_port   = 3306
    to_port     = 3306
    protocol    = "tcp"
    cidr_blocks = [var.trusted_cidr] # 수정: DB 접근을 사내망으로 제한
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = [var.trusted_cidr] # 수정: 외부 송신도 사내망으로 제한
  }
}

# -----
# 3) 사용자 DB
# -----
resource "aws_db_instance" "users" {
  identifier      = "company-users"
  engine          = "mysql"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  username        = "admin"
  password        = var.db_password # 수정: 하드코딩 대신 민감 변수 사용
  skip_final_snapshot = true

  publicly_accessible = false # 수정: 인터넷에서 DB 직접 접근 차단
  storage_encrypted    = true  # 수정: RDS 저장 데이터 암호화
  kms_key_id           = aws_kms_key.infrastructure.arn
  vpc_security_group_ids = [aws_security_group.web.id]
}

# -----
# 4) 첨부파일용 EBS 볼륨
# -----
resource "aws_ebs_volume" "attachments" {
  availability_zone = "ap-northeast-2a"
  size              = 20
  encrypted         = true # 수정: EBS 저장 데이터 암호화
  kms_key_id        = aws_kms_key.infrastructure.arn
}

```

07. 수정 후 재스캔 결과

```

timings
-----
disk i/o          361.375µs
parsing          401.459µs
adaptation        74.292µs
checks           5.83925ms
total            6.676376ms

counts
-----
modules downloaded  0
modules processed   1
blocks processed    11
files read          1

results
-----
passed            19
ignored           0
critical          0
high              0
medium            5
low               3

19 passed, 8 potential problem(s) detected.

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
-----
✅ 스캔 완료. 위에서 빨간 항목 (FAILED · CRITICAL · HIGH)을 확인합니다.
❖ (base) kimminseo@gimminseoui-MacBookAir lab %

```

심각도	수정 전	수정 후
Passed	2	19
CRITICAL	4	0
HIGH	10	0
MEDIUM	5	5
LOW	5	3
잠재 문제 합계	24	8

재스캔 결과 과제의 핵심 평가 기준인 **CRITICAL 0개, HIGH 0개**를 달성하였다. 남아 있는 MEDIUM과 LOW 항목은 S3 로깅·버전 관리, RDS 백업 보존·IAM 인증·삭제 방지, 리소스 설명 및 Performance Insights 등에 관한 추가 하드닝 권고 사항이다.

08. 한 줄 회고

가장 위험하다고 느낀 항목은 SSH와 MySQL 포트를 **0.0.0.0/0**으로 열어 둔 설정으로, 단 한 줄의 과도한 허용 규칙이 서버 침입과 데이터베이스 노출로 바로 이어질 수 있다는 점을 확인했다.

09. 결론

수정 전 4개의 CRITICAL과 10개의 HIGH 취약점을 리소스 공개 범위 축소, 저장 데이터 암호화, 비밀 정보 분리 및 고객 관리형 KMS 키 적용으로 모두 제거하였다. 또한 원본 코드의 전체 구조를 유지하여 변경 목적과 전후 차이를 쉽게 확인할 수 있도록 정리하였다.

10. 참고자료

- [tfsec: S3 Block Public ACLs](#)

- tfsec: S3 Block Public Policy
- tfsec: S3 Ignore Public ACLs
- tfsec: S3 No Public Buckets
- tfsec: S3 Enable Bucket Encryption
- tfsec: EC2 No Public Ingress Security Group Rule
- tfsec: EC2 No Public Egress Security Group Rule
- tfsec: RDS No Public DB Access
- tfsec: RDS Encrypt Instance Storage Data
- tfsec: EC2 Enable Volume Encryption
- tfsec: KMS Auto Rotate Keys

11. 보너스: 공개 IaC 저장소 스캔

11.1 스캔 대상

- 저장소: [Bridgecrew TerraGoat](#)
- 로컬 경로: `lab/bonus/terragoat`
- 선정 이유: AWS·Azure·GCP 환경의 보안 오류를 학습할 수 있도록 의도적으로 취약하게 구성된 공개 Terraform 저장소이므로 IaC 보안 스캐너의 탐지 범위를 확인하기에 적합하다.

11.2 실행 명령

```
git clone https://github.com/bridgecrewio/terragoat.git bonus/terragoat
./scan.sh tfsec bonus/terragoat
```

11.3 스캔 결과

```
timings
-----
disk i/o      5.636374ms
parsing      13.637624ms
adaptation   24.304416ms
checks       104.662209ms
total        148.240623ms

counts
-----
modules downloaded  0
modules processed   5
blocks processed    182
files read          47

results
-----
passed      51
ignored     0
critical    19
high        90
medium      89
low         48

51 passed, 246 potential problem(s) detected.

What's next:
Debug this container error with Gordon → docker ai "help me fix this container error"
-----
✅ 스캔 완료. 위에서 빨간 항목 (FAILED · CRITICAL · HIGH)을 확인합니다.
(base) kimminseo@gimminseoui-MacBookAir lab %
```

항목	결과
처리한 모듈	5
처리한 블록	182
읽은 파일	47
Passed	51
CRITICAL	19
HIGH	90
MEDIUM	89
LOW	48
잠재 문제 합계	246

11.4 결과 분석

47개의 Terraform 파일과 182개의 블록을 검사한 결과 51개 항목은 검사를 통과했지만, 총 246개의 잠재 문제가 탐지되었다. 이 가운데 즉각적인 조치 우선순위가 높은 CRITICAL과 HIGH가 각각 19개와 90개로, 두 등급을 합치면 전체 탐지 결과의 약 44.3%인 109개이다. 의도적으로 취약하게 만든 교육용 저장소임을 고려하더라도 여러 클라우드와 리소스로 구성된 실제 규모의 IaC에서는 수동 검토만으로 위험한 설정을 빠짐없이 찾기 어렵다는 점을 확인할 수 있었다.

11.5 보너스 한 줄 회고

공개 IaC 저장소를 직접 스캔해 보니 파일과 리소스가 많아질수록 취약 설정도 빠르게 누적되므로, 코드 리뷰와 함께 tfsec 같은 자동화 검사를 배포 파이프라인에 적용하는 것이 중요하다고 느꼈다.